DESIGN DOCUMENT

Custom Intrusion Detection System &
Real-Time Packet Analyzer

Group 4

24114005 Aditya Bhadoria

24114006 Aditya Ranjan

24114028 Daddi Om

24114032 Deokar Parth

24114050 Kothawade Manthan

24114088 Satyam Sharma

CSC-303 – Computer Networks

October 7, 2026

Contents

1 Summary **2**

2 System Architecture & High-Level Design **2**
2.1 Concurrency Model 2
2.2 8-Stage Processing Pipeline (DFD) 2

3 Data Dictionary & State Management **4**
3.1 The `PacketRecord` Structure 4
3.2 The `IncidentState` Structure 4

4 Low-Level Design (Algorithms) **4**
4.1 Algorithm 1: TCP Stream Reassembly & LRU Eviction 5
4.2 Algorithm 2: Sub-Bucketed Time-Window Threshold Detection 6
4.3 Algorithm 3: Multi-Stage Incident Correlation (Kill-Chain) 7

5 System Constraints & Performance Considerations **7**
5.1 Memory & Garbage Collection Tuning 7
5.2 Threat Intelligence Feed Optimization 8
5.3 Security & Least Privilege 8

6 Testing & Quality Assurance **8**

7 References **8**

1 Summary

This document details the architectural, structural, and procedural design of a Custom Intrusion Detection System (IDS) and Real-Time Packet Analyzer. Developed in Go (1.27.x), the system provides deep packet inspection across the OSI stack (L2 to L7). It is capable of capturing live network traffic, rigorously parsing headers, reassembling TCP streams, executing a custom Snort-like rule engine, and correlating multi-stage incidents.

The core design philosophy prioritizes determinism, modularity, and high throughput (capable of processing $> 250,000$ packets per second). To achieve this without introducing lock contention or packet reordering, the system relies on an 8-stage sequential processing pipeline executed primarily within a single goroutine, decoupled from the capture interface via a bounded lock-free channel. This document expands upon the High-Level Design (HLD), Data Structures, Low-Level Algorithms (LLD), and System Performance Constraints.

2 System Architecture & High-Level Design

The system architecture avoids complex multi-threading for protocol parsing to completely eliminate the risk of processing packets out-of-order, which would critically break stateful stream reassembly and time-windowed threshold detectors.

2.1 Concurrency Model

The application utilizes a **Single-Producer, Single-Pipeline Consumer** model:

- **Producer (Capture Goroutine):** Interfaces directly with `libpcap`. It reads raw bytes, applies kernel-level BPF (Berkeley Packet Filter) strings to discard irrelevant traffic at the OS level, allocates a monotonic capture timestamp, and pushes the byte slice to a bounded ring-buffer queue.
- **Consumer (Pipeline Goroutine):** Pops raw frames from the queue and passes them sequentially through stages 2 through 8. Because this is a single execution thread, the "Engine Time" advances strictly forward, ensuring that stateful algorithms (like TCP reassembly or sliding-window rate limiters) behave deterministically.

2.2 8-Stage Processing Pipeline (DFD)

The data flow follows a strict 8-stage pipeline. A single mutable `PacketRecord` object is instantiated at the start of the consumer loop and passed by reference through the subsequent stages to minimize heap allocations and garbage collection (GC) overhead.

Stage 1: Capture: Promiscuous mode capture of Ethernet frames. If the internal queue overflows due to a slow consumer (e.g., traffic spikes $> 300,000$ pps), the

capture stage drops the packet and increments an atomic `QueueDrop` counter rather than blocking the network interface.

Stage 2: Lower Parsers (L2/L3):

- *Data Link (L2):* Extracts Ethernet source/destination MAC addresses. Skips 802.1Q/802.1ad VLAN tags up to a bounded depth.
- *Network (L3):* Parses IPv4 (options, fragmentation, TTL) and IPv6 (iterating through extension headers like Hop-by-Hop, Routing, and Fragment). Extracts ARP requests/replies.

Stage 3: Upper Parsers (L4):

- *TCP/UDP:* Extracts ports, sequence numbers, acknowledgement numbers, flags, and window sizes.
- *ICMP:* Parses Type/Code and extracts original IP headers encapsulated in ICMP error messages. Validates checksums using the L3 pseudo-header.

Stage 4: Stream Reassembly: A stateful engine that buffers out-of-order TCP segments. It ignores identical retransmissions and flags overlapping segments with conflicting data as TCP Evasion attempts. Completely reconstructed application-layer messages are passed forward.

Stage 5: Application Parsers (L7): Identifies protocol by payload signature, not by port.

- *HTTP/1.x:* Parses request lines, headers, URI (percent-decoded).
- *DNS:* Unpacks questions, resolves compression pointers.
- *TLS:* Extracts Server Name Indication (SNI) and computes JA3 client fingerprints (filtering out GREASE values).

Stage 6: Rule Engine: Evaluates the fully annotated `PacketRecord` against loaded rules. Triggers built-in stateful detectors (e.g., SYN Flood, Slowloris, Land attacks).

Stage 7: Correlation: Groups isolated alerts into cohesive incidents. Uses kill-chain logic to map alerts to *recon*, *exploit*, *c2* (command and control), and *exfil* stages based on source/destination pairs.

Stage 8: Outputs: Formats alerts into JSON Lines (JSONL), managing file size rotation (e.g., max 100MB per file). Pushes aggregate metrics to the Terminal UI (TUI).

3 Data Dictionary & State Management

To achieve zero-copy packet processing where possible, the system relies heavily on Go slice references rather than byte copying.

3.1 The `PacketRecord` Structure

The `PacketRecord` is the central data structure carrying the context of a single packet as it traverses the pipeline.

Field Name	Data Type	Description
`Timestamp`	`time.Time`	Hardware or libpcap-assigned arrival time.
`RawBytes`	`[]byte`	Reference to the original captured frame.
`Eth`	`L2Context`	Parsed MAC addresses and EtherType.
`IP`	`L3Context`	Src/Dst IPs, TTL, Protocol, Fragmentation offsets.
`Transport`	`L4Context`	Src/Dst Ports, TCP Flags, Seq/Ack numbers.
`AppPayload`	`[]byte`	Reassembled stream data (if applicable).
`L7Context`	`AppMetadata`	Parsed HTTP URI, DNS Queries, or TLS JA3 hashes.
`IsSpoofable`	`bool`	Flag set to true for UDP/ICMP or incomplete TCP.

Table 1: PacketRecord Structure schema.

3.2 The `IncidentState` Structure

Used by the Correlation Engine to track multi-stage attacks over a 24-hour rolling window. State is capped at 10,000 active entities with LRU eviction to bound memory to < 256 MB.

Field Name	Data Type	Description
`AttackerIP`	`net.IP`	The reliable (non-spoofed) source of the attack.
`VictimIP`	`net.IP`	The target of the attack stages.
`HighestStage`	`int`	Current kill-chain stage (0=Recon, 1=Exploit, etc.).
`AlertChain`	`[]AlertID`	Circular buffer of the last 64 alerts for this entity.
`LastSeen`	`time.Time`	Used for 24-hour expiration and LRU tracking.

Table 2: IncidentState Correlation tracking schema.

4 Low-Level Design (Algorithms)

The following pseudo-code blocks define the deterministic, high-performance algorithms implemented within the processing pipeline.

4.1 Algorithm 1: TCP Stream Reassembly & LRU Eviction

Stream reassembly is computationally expensive. This algorithm handles out-of-order segments while strictly bounding memory usage via an LRU cache.

Algorithm 1 TCP Stream Reassembly

Require: *packet* (containing L3 and L4 Contexts), *engineTime*

$flowKey \leftarrow Hash(packet.SrcIP, packet.DstIP, packet.SrcPort, packet.DstPort)$
$flow \leftarrow StreamTable.Get(flowKey)$
if $flow == Nil$ **then**
    if $packet.Flags == SYN$ **then**
        $flow \leftarrow CreateNewFlow(packet.SeqNum)$
        $StreamTable.Insert(flowKey, flow)$
        $UpdateLRU(flowKey)$
    end if
    return Nil ▷ Cannot reconstruct without handshake
end if
$UpdateLRU(flowKey)$
if $packet.SeqNum == flow.ExpectedSeq$ **then**
    $flow.Buffer.Append(packet.Payload)$
    $flow.ExpectedSeq \leftarrow packet.SeqNum + Length(packet.Payload)$
    while $flow.OutOfOrderQueue.Has(flow.ExpectedSeq)$ **do**
        $queuedSegment \leftarrow flow.OutOfOrderQueue.Pop(flow.ExpectedSeq)$
        $flow.Buffer.Append(queuedSegment.Payload)$
        $flow.ExpectedSeq \leftarrow queuedSegment.SeqNum + Length(queuedSegment.Payload)$
    end while
    if $IsMessageBoundary(flow.Buffer)$ **then**
        $appMessage \leftarrow flow.Buffer.ExtractMessage()$
        return $appMessage$
    end if
else if $packet.SeqNum > flow.ExpectedSeq$ **then**
    $flow.OutOfOrderQueue.Insert(packet.SeqNum, packet.Payload)$
else if $packet.SeqNum < flow.ExpectedSeq$ **then**
    if $PayloadDiffers(packet.Payload, flow.Buffer.Get(packet.SeqNum))$ **then**
        TriggerAlert("TCP Evasion: Overlapping Conflicting Data")
    end if
end if
if $StreamTable.MemoryUsage > MaxMemoryCap$ **then**
    $StreamTable.EvictOldestLRU()$
end if
return Nil

Complexity & Implementation Notes for Algorithm 1:

- **Time Complexity:** $O(1)$ expected for in-order packets. $O(\log K)$ for out-of-order segment insertion where K is the current gap size, implemented using a Red-Black Tree or Min-Heap for the out-of-order queue.

- **Space Complexity:** Bounded strictly to `MaxMemoryCap` (e.g., 64 MiB). Once the cap is hit, the $O(1)$ LRU doubly-linked list evicts the oldest inactive flow to prevent resource exhaustion attacks.

4.2 Algorithm 2: Sub-Bucketed Time-Window Threshold Detection

For volumetric attacks like UDP Floods or ICMP Smurf attacks, maintaining a distinct timestamp for every packet is inefficient. Instead, the detection engine uses a sub-bucketed sliding window.

Algorithm 2 Sub-Bucketed Rate Limiting (e.g., UDP Flood)

Require: $source$, $packetLength$, $engineTime$, $thresholdBytes$, $windowSecs$

$numBuckets \leftarrow 10$ ▷ Provides 10% boundary accuracy
$bucketDuration \leftarrow windowSecs / numBuckets$
$state \leftarrow FloodTable.Get(source)$
$currentBucketIndex \leftarrow (engineTime / bucketDuration) \pmod{numBuckets}$
if $state.LastUpdateBucket \neq currentBucketIndex$ **then**
    $bucketsToClear \leftarrow CalculatePassedBuckets(state.LastUpdateTime, engineTime)$
    for $i = 1$ **to** $\min(bucketsToClear, numBuckets)$ **do**
        $idx \leftarrow (state.LastUpdateBucket + i) \pmod{numBuckets}$
        $state.TotalBytes \leftarrow state.TotalBytes - state.Buckets[idx]$
        $state.Buckets[idx] \leftarrow 0$
    end for
    $state.LastUpdateBucket \leftarrow currentBucketIndex$
end if
$state.Buckets[currentBucketIndex] \leftarrow state.Buckets[currentBucketIndex] + packetLength$
$state.TotalBytes \leftarrow state.TotalBytes + packetLength$
$state.LastUpdateTime \leftarrow engineTime$
if $state.TotalBytes \geq thresholdBytes$ **then**
    if $TimeSinceLastAlert(source) > DeduplicationWindow$ **then**
        TriggerAlert("Volumetric Flood Detected")
    end if
end if

Complexity & Implementation Notes for Algorithm 2:

- **Time Complexity:** $O(1)$ time per packet. The loop to clear stale buckets runs at most 10 times, resolving in constant time.
- **Space Complexity:** $O(1)$ per tracked IP. By using 10 integer counters rather than a queue of thousands of timestamps, the system handles millions of packets per second per IP without memory bloat.

4.3 Algorithm 3: Multi-Stage Incident Correlation (Kill-Chain)

This algorithm evaluates individual alerts and aggregates them into high-severity incidents if they fit a sequential attack pattern (Recon → Exploit → C2 → Exfiltration).

Algorithm 3 Incident Correlation Engine

Require: $alert$, $engineTime$
if $alert.SourceIsSpoofable$ **then**
    return ▷ Never correlate spoofable UDP/ICMP sources as reliable attackers
end if
$incidentKey \leftarrow Hash(alert.AttackerIP, alert.VictimIP)$
$incident \leftarrow IncidentTable.GetOrCreate(incidentKey)$
$incident.AlertChain.PushBack(alert)$
$stage \leftarrow MapAlertToKillChainStage(alert.Category)$
if $stage > incident.HighestStage$ **then**
    if $engineTime - incident.LastStageTime \leq 24hours$ **then**
        $incident.HighestStage \leftarrow stage$
        $incident.LastStageTime \leftarrow engineTime$
        if $incident.HighestStage \geq 2$ **then**
            EscalateSeverity("Multi-Stage Incident", CRITICAL)
            EmitIncidentUpdate(incident)
        end if
    end if
else if $stage == EXPLOIT$ **and** $alert.Category == C2_BEACON$ **then**
    if $CheckCallback(alert.VictimIP, alert.AttackerIP)$ **then**
        TriggerAlert("Compromised Host: Callback Detected")
    end if
end if

5 System Constraints & Performance Considerations

To ensure the system meets its Non-Functional Requirements (NFRs), particularly NFR-PERF-1 ($> 250,000$ packets/s) and NFR-MEM-1 ($\leq$ 256 MB peak memory under a spoofed SYN flood), several deep architectural optimizations are mandated.

5.1 Memory & Garbage Collection Tuning

Go's Garbage Collector (GC) can introduce latency spikes if the heap is flooded with short-lived objects (e.g., allocating a new struct for every packet).

- **sync.Pool:** Short-lived structures used during application-layer parsing (e.g., HTTP header maps) are pooled and reused.
- **Bypassing Interface Allocations:** The `PacketRecord` does not use `interface{}` types for protocol metadata. Concrete structs are heavily utilized to keep data on the stack or inline within the `PacketRecord` to prevent heap escape.

5.2 Threat Intelligence Feed Optimization

The system must match domains, IPs, and JA3 TLS fingerprints against threat feeds containing $> 100,000$ entries at line rate.

- **IP/CIDR Matching:** Implemented using a highly optimized **Radix Tree (Trie)**. This allows a packet's IP to be matched against thousands of malicious subnets in $O(w)$ time, where w is the IP address width (32 bits for IPv4, 128 bits for IPv6), completely independent of the number of feed entries.
- **Domain/JA3 Matching:** Implemented using flat Go maps (hash tables) providing $O(1)$ expected lookup time.

5.3 Security & Least Privilege

As an IDS, the system is a high-value target for evasion and exploitation.

- **Safe Matching:** All regular expressions utilized in the rule engine are processed using the `RE2` library. Unlike PCRE engines, `RE2` guarantees linear time execution $O(N)$ and is immune to Regular Expression Denial of Service (ReDoS) attacks. Furthermore, string input to the regex engine is strictly capped at 16 KiB.
- **Capability Drops:** Instead of running the entire application as `root`, the compiled binary requires only the `CAP_NET_RAW` and `CAP_NET_ADMIN` file capabilities to access the NIC.

6 Testing & Quality Assurance

The system relies on determinism. Because "Engine Time" is derived entirely from packet capture timestamps (NFR-REL-3), replaying a `.pcap` file through the pipeline will produce byte-for-byte identical JSONL alert logs on every execution.

- **Scenario Harness:** Over 100 deterministic scenarios are documented. Each detector includes a "must fire" scenario (a PCAP of an attack) and a "must not fire" scenario (a PCAP of benign traffic that looks similar, such as high-volume NAT traffic mimicking a SYN flood).
- **Fuzzing:** Native Go fuzzing is applied to all L2-L7 parsers, ensuring that malformed inputs (e.g., overlapping TCP fragments, overlong UTF-8 URIs, truncated DNS queries) do not result in panics or index-out-of-bounds exceptions.

7 References

- RFC 791 (IPv4), RFC 8200 (IPv6), RFC 9293 (TCP), RFC 768 (UDP), RFC 792 / 4443 (ICMP).

- RFC 1034/1035 (DNS), RFC 9110/9112 (HTTP/1.1), RFC 8446 (TLS 1.3).
- Ptacek, T. H., & Newsham, T. N. (1998). *Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection*. Secure Networks, Inc.
- IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018 (Requirements Specification Structure).